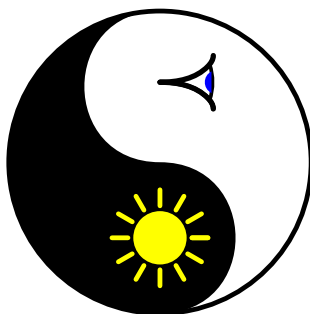


LABORATIONSUPPGIFTER
CT2890 — DATORGRAFIK, PERIOD 2, 2003/04



INLEDNING	2
LAB 1: SVEPKONVERTERING AV TRIANGLAR	4
LAB 2: 3D-TRANSFORMATIONER	6
LAB 3: SYNLIGA YTOR	10
LAB 4: BELYSNING	12
LAB 5: TEXTURMAPPNING	14

KURSINFORMATION:
<http://www.idt.mdh.se/kurser/ct2890/>

0. Inledning

Detta kompendium innehåller laborationsmaterial för kursen Datorgrafik. Syftet med laborationerna är att skriva en sk renderer, ett program som skapar syntetiska bilder utifrån geometriska beskrivningar. Programmet kommer att utvecklas stegvis så att ni efter varje laboration kan producera successivt mer realistiska bilder.

OpenGL är ett grafikbibliotek som blivit en industristandard. Vi kommer att bekanta oss med OpenGL och dess paket genom att först se hur varje laboration ser ut, skrivet i OpenGL. När ni sedan skriver er egen implementation av laborationen kommer ni att få göra det mesta av jobbet själva, men i botten använder ni OpenGL för att öppna fönster, välja färger och sätta pixlar.

Programmeringsmiljön

All programutveckling är tänkt att ske på UNIX-system, förmodligen en Linux-installerad PC. Programmering sker i grupper om högst två personer.

Det är viktigt att ni skapar en *ny katalog för varje laboration*. Börja därför redan idag med att skapa en katalogstruktur från en lämplig del av er hemkatalog, t ex

```
$ mkdir 1 2 3 4 5
```

Efter att laboration 2 är avslutad kan filerna kopieras över till katalog 3, osv. Rör sedan inte de gamla filerna eftersom datumstämpeln används vid redovisningen; mer om detta senare.

Eftersom OpenGL numera är portat till de flesta plattformar, Windows och Mac inkluderat, så kan programmen förmodligen utvecklas i andra miljöer än UNIX, men det finns inga garantier för att assistenten kan hjälpa dig om fel uppstår. Kom dock ihåg att all redovisning sker på skolan och att det inte är säkert att OpenGL finns på alla maskiner.

Deadlines

Eftersom förvärvade poäng från laborationerna viktas in i betyget kommer vi att använda deadlines för att kunna sammanställa resultatet efter tentamen. Tiderna för dessa deadlines kommer att publiceras på kursens hemsida, men i snitt kommer ni att ha två veckor till förfogande per laboration. Ju tidigare ni startar med en laboration, desto större är era chanser att hinna med en bonusuppgift. Gör dock alltid

klart de grundläggande funktionerna innan ni börjar med bonusuppgifterna!

Vad beträffar bonusuppgifter har vi redan föreslagit ett antal per laboration. Onekligen kommer några av er att tänka ut andra utökningar som ger effektivare kod, vackrare bilder eller ett mer användbart gränssnitt. Se bara till att ni förhandlar antalet poäng med examinatorn innan ni sätter igång att implementera.

Vid redovisning kommer vi att titta på filens datumstämpel. Det går alltså att *redovisa* några dagar efter en deadline, så länge den exekverbara filens datumstämpel visar att ni inte arbetat med filen efter deadline. Försök inte ändra datumstämpeln eftersom det kommer att betraktas som en form av fusk. Kom också ihåg att vissa typer av filöverföringar, tex FTP, *inte* bevarar datumstämpeln.

Följande ursäkter är inte acceptabla:

- ”Servern var nere, så jag kunde inte arbeta”.
Maskiner går alltid ner timmarna före en deadline. Undvik att hamna i den här situationen genom att börja i god tid.
- ”Alla datorer i labsalen var upptagna”.
Detta ska inte inträffa om alla följer angivna tider för respektive labsalar. Naturligtvis kommer beläggningen att öka när deadline närmar sig, men, som sagt, undvik att hamna i den här situationen genom att börja i god tid.
- ”Jag råkade ta bort den exekverbara filen”.
Förhoppningsvis är källkoden kvar, med intakt datumstämpel. Råkar ni tafs i den kan vi tyvärr inte hjälpa er, såvida inte systemadministratören kan återställa en backup.
- ”Jag missade bussen”, ”Jag hade mycket jobb med en annan kurs”, etc.
Vi kan inte göra undantag för små eller stora naturkatastrofer, men ni har möjlighet att lämna in laborationen en aning sent, dock med ett visst poängavdrag som följd. Vid sjukdom måste läkarintyg uppvisas.

Om ni måste lämna in en laboration efter angiven deadline så kommer poäng att dras av efter följande mall:

- < 2 dagar sent: 10% avdrag på poängen.
- < 4 dagar sent: 25% avdrag på poängen.
- > 4 dagar sent: ingen poäng.

Redovisning

Vid redovisning ska en hophäftad listning av ert program tillsammans med en rapport lämnas in i samband med demonstrationen för assistenten. Rapporten ska beskriva de funktioner som implementerats och vars poäng ni gör anspråk på. Under demonstrationen skall ni vara beredda på att svara på frågor om hur programmet är konstruerat.

För rapport och kod gäller följande självklara riktlinjer:

- Häfta ihop alla papper och skriv namn, uppgift och datum på framsidan.
- Dokumentera och strukturera koden på ett ingenjörsmässigt sätt. Vi kommer inte att granska obegripliga program, oavsett hur väl de fungerar!

Till sist

Se till att läsa kursens hemsida med jämna mellanrum för eventuella meddelanden:

<http://www.idt.mdh.se/kurser/ct2890/>

Vi är naturligtvis tacksamma för synpunkter på kursmaterialet. Lämna gärna sådana till mig (kpt@mdh.se) eller kursassistenten under kursens gång.

Med detta sagt hoppas vi att ni får mycket nöje av kursen!

Kjell

1. Svepkonvertering av trianglar

Förberedelser. Studera avsnittet om svepkonvertering av polygoner i föreläsningsanteckningarna samt avsnitt 3.6 i läroboken. Läs alltid igenom hela texten innan ni börjar med laborationen!

Uppgift. Er uppgift är att skriva en rutin som svepkonverterar trianglar enligt de principer som presenterats i boken och på föreläsningen.

Algoritmen kommer att användas i senare laborationer för framställning av tredimensionella objekt. Därför är det viktigt att er implementation är lätt att förstå så att ni enkelt kan lägga till kod senare.

Till er hjälp har ett labskelett med drivrutiner skrivits som läser in trianglarnas koordinater från en fil och anropar er rutin. Detta labskelett finns att hämta på kursens hemsida och använder OpenGL:s triangelfyllare för att visa hur programmet är tänkt att fungera. Studera koden, skriv `make` för att kompilera och testkör sedan programmet.

Grundläggande funktioner. För att erhålla full poäng på laborationen måste ni implementera de grundläggande funktionerna som beskrivs nedan (50 poäng). Om ni vill kan ni dessutom genomföra några av de valfria alternativen för att skaffa extra poäng.

Vid redovisning av delmomentet *Glipor och överlappning* (se nedan) gäller följande förmildrande regel: om glipor eller överlappning förekommer vid redovisningen så har man möjlighet att hitta felet och redovisa igen, men då endast för 8 poäng. Finns fortfarande glipor/överlappning så har man ytterligare en sista chans, då för 4 poäng.

Spara för säkerhets skull den version som klarar baskraven innan ni ger er på bonusuppgifterna, dels för att ni ska ha något att visa upp om ni inte hinner klart, dels för att någon av finesserna kanske inte passar ihop med resten av laborationerna.

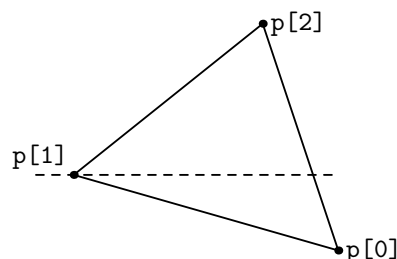
- **Svepkonvertering** (20 p).

Funktionen `scan3(point p)`, där `p[0...2]` definierar triangelns hörn, använder för närvarande OpenGL:s triangelfyllare. Ersätt OpenGL-anropen med egen kod. Ni kan använda `glVertex2i(x,y)` för att sätta en punkt på skärmen i position (x,y) ¹. Som ni kanske räknat

¹Av implementationstekniska skäl rekommenderas OpenGL-programmerare normalt att addera 0.375 till x - och y -koordinater för pixeladressering. Detta för att undvika problem med trunkering. Av detta skäl innehåller labskelettet ett anrop till `glTranslatef()` som flyttar koordinatsystemet 0.375 enheter i både x - och y -led.

ut behöver man inte implementera den generella algoritmen för svepkonvertering av polygoner eftersom trianglar är ett ganska enkelt specialfall. Det lönar sig dock att förstå denna algoritm innan ni tar fram yxan.

Implementera inte någon konstig algoritm – det leder bara till problem senare när ni ska utvidga svepkonverteraren. Ni kan t ex istället utnyttja det faktum att en generell triangel kan delas längs en horisontell linje så att två enklare trianglar bildas (fig 1). Dessa trianglar kan sedan fyllas i horisontella rader.



Figur 1: Godtycklig triangel uppdelad i två enklare.

Om ni utgår från algoritmen från föreläsningen, tänk då på att den aktiva kanttabellen (AET) alltid kommer att innehålla exakt två kanter.

Det är fullt acceptabelt att använda flyttal för att hålla reda på kanternas x -koordinater och lutning. En extrauppgift är att implementera en heltalsbaserad variant.

- **Förkasta urartade trianglar** (5 p).

Er svepkonverterare ska detektera och förkasta urartade trianglar (två eller tre sammanfallande punkter). Returnera `-1` från `scan3` när detta inträffar.

- **Effektiv kod** (10 p).

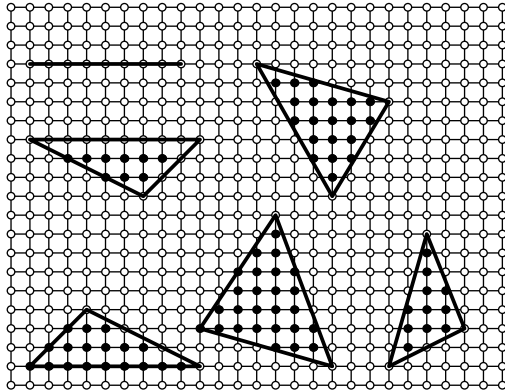
Examinatorn kan komma att köra ert program med stoppur så var beredd på att rita tusentals trianglar. En långsam implementation kan ge poängavdrag; försök därför välja datastrukturer som gör att programmet löper så fort som möjligt.

- **Glipor och överlappning** (15 p).

Angränsande trianglar (i ett lapptäck) ska passa ihop perfekt utan glipor eller överlappning. För att verifiera detta kommer drivrutinen att rita trianglarna i XOR-mod vilket innebär att både

glipor och överlappningar kommer att synas tydligt. Ett bra testfall är en paj med triangelformade tårtbitar.

För att lyckas är det viktigt att tillämpa konsekventa regler för pixels som ligger nära eller på kantlinjerna. Det är inte tillåtet att ”komma ihåg” vilka pixels ni satt – de regler ni implementerar ska bara beröra den triangel som ritas för tillfället.



Figur 2: Några testfall. Oändligt tunna trianglar (överst till vänster) leder inte till att några pixlar sätts.

Tips. Utöver de ledtrådar som givits ovan kommer här några andra praktiska tips.

- *Strimlor.* Långsmala trianglar kan ge upphov till strimlor (slivers”). Försök inte hantera dessa.
- *Verktyg.* Använd UNIX-programmet `xmag` eller `printf`-satser för att se vilka pixels ni satt.
- *Avlusning.* Testa programmet noga genom att mata programmet med alla möjliga vidriga trianglar. Några trianglar kommer att läggas upp på kursens hemsida men det är mest för att visa formatet på filen. Under demonstrationen kommer betydligt elakare testfall att användas.

Extra funktioner. Bonuspoäng kan införskaffas genom att lösa några av följande uppgifter.

- *Heltalsbaserad variant* (10 p).

Att koda svepkonverteraren med endast heltal är lite knepigare, men föreläsningsmaterialet ger en del ledtrådar. Var noggrann med att testa alla kombinationer av lutningar om ni väljer denna uppgift.

- *Svepkonvertering av konvexa polygoner* (5 p).

Skriv en preprocessor som fyller konvexa polygoner genom att dela upp dessa i trianglar. En lämplig algoritm för detta är följande: låt $P_1 \dots P_n$ vara hörnpunkter i den konvexa polygonen. Dra nu linjer från P_1 till P_3 , P_1 till P_4 , $\dots$, P_1 till P_{n-1} .

- *Svepkonvertering av polygoner* (20 p).

Implementera den algoritm för svepkonvertering av godtyckliga polygoner – både konvexa och konkava – som diskuterats på föreläsningen.

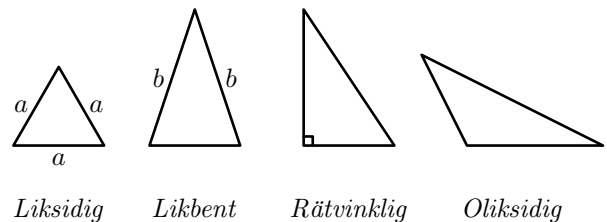
- *Klippning av trianglar* (20 p)

Anpassa Sutherland och Hodgmans polygonklipp-algoritm för trianglar som ett försteg i er svepkonvertering. Ni behöver bara klippa efter ett rektangulärt fönster.

- *Egna initiativ* (? p)

Kom ihåg att ni kan hitta på egna utökningar. Diskutera dock med examinatorn först så ni vet hur många poäng uppgiften är värd.

Ordlista: För att förenkla diskussionen mellan dig, din labpartner och assistenter så återges namnen på några olika trianglar här.

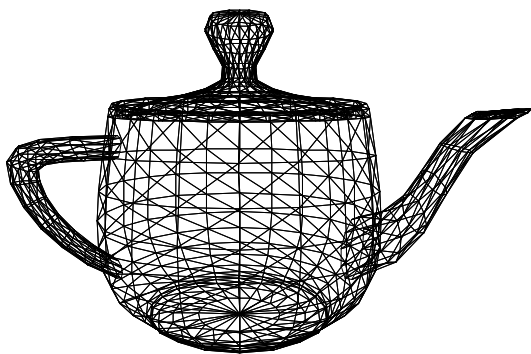


2. 3D-transformationer

Förberedelser. Repetera föreläsningsanteckningarna för kapitel 6. Eftersom boken använder PHIGS-nomenklatur är ni inte så betjänta av kursboken för denna laboration.

Uppgift. Uppgiften består i att skriva ett program som implementerar 3D vytransformationer i en värld av objekt uppbyggda av triangelytor (fig. 3). Programmet ska visa objekten som genomskinliga trådmodeller. Efter tillägg av Z-buffert, belysning och texturmappling i efterföljande laborationer kommer ni att ha en fullständig "rendering pipeline" som klarar av att generera realistiska (näja...) 3D-scener.

Till ert förfogande finns en OpenGL-version av laborationen med rutiner för bl a inläsning av objekt i sk PLG-format samt ett paket för matriser och vektorer. Skriv `make` för att kompilera programmet och t ex `render box.plg` för att titta på `box.plg`. Er uppgift blir att byta ut de flesta OpenGL-anropen mot egen kod.



Figur 3: Trådmodell för en tekanna: `pot.plg`.

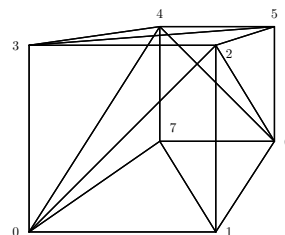
PLG-formatet. På hemsidan finns ett antal PLG-filer. Varje sådan fil består av tre delar: ett huvud, en lista av noder samt en lista av (konvexa) polygoner (fig. 4). När en PLG-fil laddas in av labskelettet delas polygonerna automatiskt upp i trianglar.

Huvudet i PLG-filen består av objektets namn, antalet noder samt antalet polygoner. Efter denna rad följer noderna – en x, y, z -trippel per rad. Noderna numreras från 0 och uppåt. Slutligen följer polygoninformationen, en polygon per rad på formen

materialbeskrivning $n \ v_1 \ v_2 \ \dots \ v_n$

Materialbeskrivningen används inte i denna kurs. Resten av raden innehåller n , antalet hörnpunkter, samt de nodnummer som ingår i polygonen.

```
BOX 8 6
0 0 0
200 0 0
200 200 0
0 200 0
0 200 200
200 200 200
200 0 200
0 0 200
0x11FF 4 0 1 2 3
0x11FF 4 4 5 6 7
0x11FF 4 7 6 1 0
0x11FF 4 6 5 2 1
0x11FF 4 5 4 3 2
0x11FF 4 4 7 0 3
```



Figur 4: PLG-fil för en kub: `box.plg`.

Labskelettet. Ladda ner och packa upp den gzipade tar-filen med labskelettet från kursens hemsida.

1. Modulen `algebra.h` definierar tre datatyper,

```
typedef struct { GLdouble x,y,z; } Vector;
typedef struct { GLdouble x,y,z,w; } HomVector;
typedef struct { GLdouble e[4][4]; } Matrix;
```

För vektorer u och v , matriser P och Q samt skalär t finns här operationer för bl a $u + v$, $u - v$, $u \times v$, $u \cdot v$, $|u|$, tu , Pu och PQ . Bekanta er med koden i `algebra.c` så att ni vet vad varje rutin gör. Som ni ser arbetar paketet normalt med vektorer av typen `Vector` men multiplikation av en matris med en vektor resulterar i en `HomVector`, dvs en homogen vektor. Detta beror på att klippning helst utförs i homogena koordinater (se boken och föreläsningsanteckningar). I alla andra fall är det enklare och snabbare att arbeta med 3D-vektorer av typen `Vector`.

Några operationer saknas, bl a generering av matriser för rotation, skalning, translation och perspektivtransformation. En av uppgifterna i denna laboration är att skriva klart dessa.

2. Modulen `object.h` definierar datatypen `Obj` som definierar ett objekt uppbyggt av triangelytor. Ett objekt består av följande information.

```
typedef struct obj {
    int nv;
    Vertex *vertices;
    int nt;
```

```

Triangle *triangles;
struct obj *next;
} Obj;

```

Dessa fält innehåller följande information:

- **nv**: antalet noder (eng. *vertices*) i objektet.
- **vertices**: en lista av nv st noder.
- **nt**: antalet trianglar i objektet.
- **triangles**: en lista av nt st trianglar.
- **next**: en pekare till nästa objekt.

Fig. 5 visar datastrukturen för kuben i fig. 4.

En nod (eng. *vertex*) definieras av

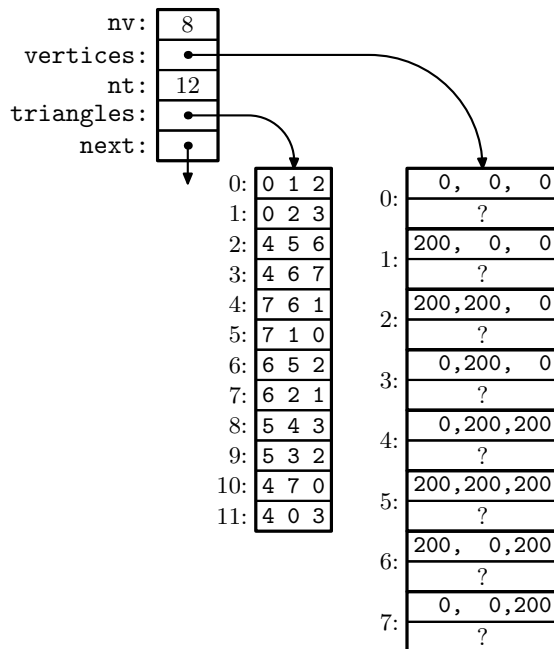
```

typedef struct vertex {
    Vector world;
    Vector screen;
} Vertex;

```

Fältet **world** kommer, efter det att objektet laddats in, att innehålla nodens position i världen. När trianglar sedan skickas genom transformationskedjan ska **screen**, dvs skärmpositionen, räknas ut.

En **Triangle** är definierad som tre st index till **vertices**.



Figur 5: Datastruktur för kuben.

Inladdning av objekt från PLG-filer sker med **load(char *filename)**. Labskelettet är redan skrivet så att alla argument på kommandoraden laddas in och placeras i en länkad lista **objlist**.

Ett användbart makro för åtkomst av noder i ett objekt finns också definierat i **object.h**: **VERTEX(obj, t, i)** ger den nod som finns i triangel **t**, hörn **i** ($0 \leq i \leq 2$).

3. Modulen **render.c** innehåller **main** som laddar in alla objekt, startar OpenGL och slutligen anropar **glutMainLoop()**. När en tangent trycks ned anropas funktionen **keys()** som använder följande tangenter för att låta observatören flytta och vrida sig i världen:

Öka	Minska	
X	x	observatörens <i>x</i> -koordinat.
Y	y	observatörens <i>y</i> -koordinat.
Z	z	observatörens <i>z</i> -koordinat.
i	k	vinkel mot <i>x</i> -axeln ("pitch").
j	l	vinkel mot <i>y</i> -axeln ("yaw").

Efter varje tangentnedtryckning anropas **glutPostRedisplay()** som gör att funktionen **display()** körs. Denna funktion är tänkt att rita vad observatören "ser" genom att stega igenom objektlistan och anropa **wire()** för varje objekt.

Grundläggande funktioner. För att erhålla full poäng på laborationen måste ni implementera de grundläggande funktionerna som beskrivs nedan (50 poäng). Om ni vill kan ni sedan genomföra några av de valfria alternativen.

- **Matrispaketet** (15 p).

Komplettera **algebra.c** med rutiner som genererar matriser för rotation, skalning, translation och perspektivtransformation. I OpenGL utförs dessa operationer på den matris som är aktuell, vilket ställs in med **glMatrixMode()**. Detta har att göra med att så gott som alla OpenGL-anrop har hårdvarustöd. Vi kommer istället att skicka runt matriserna som argument, vilket gör det lite mer lättarbetat och generellt. Några exempel följer. P är här perspektivtransformationsmatrisen och V modelview-matrisen.

- OpenGL:

```

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(fov, 1.0, near, far);

```

- Laboration 2:

```
P = Perspective(fov, near, far);
```

(Eftersom vi ritar i ett kvadratisk fönster kommer den s k *aspect ratio* att alltid vara 1.0 och kan därmed slopas.)

- OpenGL:

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glRotated(angle, 1, 0, 0);
glTranslated(dx, dy, dz);
```

- Laboration 2:

```
V = RotX(angle);
V = MatMatMul(V, Translate(dx, dy, dz));
```

(Vi har valt att ersätta OpenGL:s generella `glRotated()`-funktion med tre st rotationsfunktioner, en för varje axel.)

Observera att vektorer och matriser skickas "by-value" eftersom de är inneslutna i en `struct`.

- *Vyspecifikation* (15 p).

Implementera den vyspecifikation som behandlats på föreläsningarna.

- Perspektivtransformationsmatrisen definieras av lämpliga värden på synfältets vinkel, samt avstånden till främre och bakre klipplan.
- Modelview-matrisen definieras av observatörens position och orientering.

- *Transformation och klippning* (20 p).

Skicka era trianglar genom den 6-stegs pipeline som diskuteras på föreläsningen. Utför helst bara en matrismultiplikation per nod genom att multiplicera ihop alla transformationsmatriser i förväg. Poängavdrag kommer att göras för onödiga matrismultiplikationer.

Om ett triangelhorn faller utanför synfältet kan ni strunta i den triangeln. (OpenGL kommer inte att klippa korrekt när ni väl hamnat i 2D utan man måste istället klippa efter vyvolymen vilket är en extrauppgift.) Villkoren för att förkasta en triangel finns beskrivet i boken och i anteckningarna. Observera att homogen representation används!

När skärmkoordinaterna räknats ut för de synliga trianglarna kan ni koppla ihop dessa med `glBegin(GL_LINE_LOOP) ... glEnd()`.

Extra funktioner.

- *Styrning efter syftlinjen* (10 p)

Helst skulle man vilja låta observatören förflytta sig i synfältets riktning och inte bara efter axlarna. Implementera denna funktion, men kom ihåg att sammansättning av 3D rotationsmatriser inte är kommutativ. Exempel:

$$R_y(180^\circ)R_x(90^\circ) \neq R_x(180^\circ)R_y(90^\circ)$$

- *Musbaserat användargränssnitt* (5–10 p)

Många 3D-ritprogram använder musen för att navigera runt i rymden. Kanske ni har använt 3D Studio eller liknande system och tycker det vore intressant att införa det även i ert program.

Ett minimikrav i den här uppgiften är att användaren ska kunna navigera runt ett objekt med hjälp av musen utan att använda tangentbordet.

- *gluLookAt* (10 p)

GLU-paketet har en funktion `gluLookAt`. Läs mer om hur denna funktion fungerar och implementera er egen `LookAt`-funktion. Programmet `projection` från SIGGRAPH 99:s "Interactive OpenGL" finns på kursidan om ni vill se ett exempel. Demonstrera för assistenten genom att "flyga" över tekannen genom att successivt trycka på någon tangent.

- *3D-klippning* (20 p)

Om en triangel har hörn utanför synfältet men åtminstone ett inuti, kan man klippa triangeln efter vyvolymens begränsningsplan.

Man kan med fördel använda Sutherland-Hodgmans algoritm för polygonklippning även här, dock med skillnaden att det nu finns sex klipplan (bak, fram, vänster, höger, under, över) och att klipptesten måste formuleras i homogena koordinater.

Observera att efter klippningen kan triangeln bli en n -hörning. Hur stort kan n bli?

Artikeln "Clipping using homogenous coordinates" av Blinn & Newell rekommenderas för denna uppgift.

- *Parallellprojektion* (10 p)

Läs mer om GL-funktionen `glOrtho()` och prova hur den fungerar i GL-versionen av lab 2. Skriv en variant av `glOrtho()` till ert eget program.

Vilket värde får projektionsmatrisen?

- *Egna initiativ* (? p)

Kom ihåg att ni kan hitta på egna utökningar, även för OpenGL-versionen. Dvs, ni kan implementera något intressant som ni läst om i OpenGL-guiden.

Diskutera dock med examinatorn först så ni vet hur många poäng uppgiften är värd.

Tips. För att underlätta felsökandet visas här hur matriserna S , P , V och M ser ut för följande värden på vypyramiden och observatören i $(0, 0, 500)$.

$$\begin{aligned}\text{fov} &= 50^\circ \\ \text{near} &= 5.0 \\ \text{far} &= 10000.0\end{aligned}$$

$$S = \begin{pmatrix} 350 & 0 & 0 & 350 \\ 0 & 350 & 0 & 350 \\ 0 & 0 & 700 & 700 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$P = \begin{pmatrix} -2.15 & 0 & 0 & 0 \\ 0 & -2.15 & 0 & 0 \\ 0 & 0 & -1 & -5 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$V = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -500 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M = SPV = \begin{pmatrix} -750.6 & 0 & 350 & -175000 \\ 0 & -750.6 & 350 & -175000 \\ 0 & 0 & -0.35 & -3326.67 \\ 0 & 0 & 1 & -500 \end{pmatrix}$$

3. Synliga ytor

Förberedelser. Läs föreläsningssanteckningarna för kapitel 13 och studera bokens avsnitt om *back-face culling* (13.1.4) samt Z-buffert (13.2).

Uppgift. Nästa steg är att undanröja de triangelytor som observatören omöjligen kan se. Dessa ytor kan antingen vara *skymda* eller utgöra *baksidor*. Baksidor kan elimineras snabbt och effektivt genom att studera den projicerade triangelns normal – om den pekar bort från observatören kan triangel kastas. Detta eliminerar i regel hälften av alla trianglar; för resterande (eventuellt skymda) ytor skall ni använda en Z-buffert.

Grundläggande funktioner. För att erhålla full poäng på laborationen måste ni implementera de grundläggande funktionerna som beskrivs nedan (50 poäng). Inget nytt labskelett behövs till denna laboration, men en *Makefile* som kopplar in svepkonverteraren finns att hämta på kursens hemsida. Det finns dessutom en OpenGL-version av denna laboration som ni kan ladda ner och köra i jämförande syfte.

- *Baksidor* (20 p).

Eliminera baksidor i er pipeline genom att undersöka de projicerade trianglarnas normaler. Försök att göra detta test så effektivt som möjligt, så som diskuterats på föreläsningen.

Verifiera att koden fungerar med trådmodeller innan ni börjar med nästa del.

- *Z-buffert* (30 p).

Koppla nu in er svepkonverterare från laboration 1 och ändra `scan3` till

```
#include "object.h"
int scan3(Vertex p[]) { ... }
```

där `p[0...2].screen` är hörnpunkternas skärmkoordinater för den triangel som just passerat er pipeline.

Installera sedan en Z-buffert i svepkonverteraren. Interpolera z -värdet från hörnpunkternas z -värden.

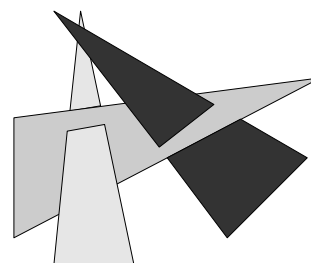
Eftersom vi inte gått igenom belysning ännu kan ni färglägga ytorna slumpaktigt så att objekten inte avbildas som en enda stor silhuett. Gör t ex följande anrop innan ni ritar en triangel.

```
glColor3f(rand() / (double) RAND_MAX,
           rand() / (double) RAND_MAX,
           rand() / (double) RAND_MAX);
```

Om programmet fungerar på `penetrate.plg` (fig. 6) har ni förmodligen gjort någonting rätt. (Placera observatören i (0,0,1700) för bästa utsikt.)

PENETRATE 9 3

```
100 0 0
300 0 0
200 700 80
0 100 40
800 500 50
0 400 40
600 100 80
800 300 80
100 600 0
0 3 0 1 2
0 3 3 4 5
0 3 6 7 8
```



Figur 6: Filen `penetrate.plg`.

Extra funktioner.

- *Inladdning av en hel värld* (10 p)

Problemet med PLG-filerna är att alla objekt har sitt origo i världskoordinater (0,0,0). Uppgiften i denna funktion är att ersätta koden i `main` som laddar in PLG-filen,

```
for (i = 1; i < argc; i++) {
    obj = load(argv[i]);
    obj->next = objlist;
    objlist = obj;
}
```

med ett funktionsanrop `load_world(argv[1])`, där `argv[1]`, argumentet till `render`, är en sk *world*-fil där varje rad har följande utseende:

```
object PLG-fil  $s_x, s_y, s_z$   $\theta_x, \theta_y, \theta_z$   $x_w, y_w, z_w$ 
:
```

där $s_{x,y,z}$ anger skalfaktorer, $\theta_{x,y,z}$ orientering och (x_w, y_w, z_w) är objektets position i världen. Skriv funktionen `load_world(char *filename)`. Använd algebra-paketet för att skala, vrida och

förflytta. Glöm inte bort att länka in varje objekt i listan `objlist`.

Som körexempel finns en tar-fil på kursens hemsida med ett schackspel, `chess.wld`.

- *3D-picking* (10 p)

Fundera ut ett sätt att låta användaren klicka på och markera ett objekt. Visa på något sätt att det utvalda objektet markerats, t ex genom att rita det med en speciell färg. Ledtråd: använd en buffert liknande Z-bufferten.

- *Undvik att rensa Z-bufferten* (10 p)

Fundera ut en sätt att slippa rensa Z-bufferten före varje scen.

- *Undvik sågtandseffekten* (5 p)

Som ni kanske märkt ger `penetrate.plg` upphov till sågtandsliknande kanter från vissa vyer. Fundera ut varför det blir så. Om ni dessutom korrigerar för detta fås ytterligare 5 p.

- *Mer bonusuppgifter?*

Kom ihåg att ni kan hitta på egna utökningar, även för OpenGL-versionen.

4. Belysning

Förberedelser. Repetera avsnitt 14.1 och 14.2 i kursboken samt motsvarande avsnitt i föreläsninganteckningarna.

Uppgift. Vi kommer i denna laboration att implementera OpenGL:s "Phong-lighting" vilket i grund och botten är en Gouraud-shader, dvs intensiteten beräknas för hörnpunkterna och interpoleras sedan över triangeln. Till skillnad från en traditionell Gouraud-shader tas även komponenten för speglande reflektion med i beräkningarna vilket gör att man kan få "blänk" på ytorna om trianglarna är tillräckligt små.

Det finns en "nedbantad" OpenGL-version av denna laboration att ladda ner från hemsidan. Den visar tekannan, men klarar inte godtyckliga PLG-filer eftersom det skulle avslöja lösningen till den första uppgiften. Tekannan ritas för övrigt med GLUT-funktionen `glutSolidTeapot()` som emellertid verkar vara definierad in-och-ut.

Vi kommer att använda gråskalor (färg är en bonusuppgift) vilket innebär att när ni räknat ut en pixels intensitet $i \in [0,1]$ kan ni använda `glColor3f(i, i, i)` innan ni sätter pixeln.

Vi måste sedan bestämma oss för huruvida vi vill visa "flata" objekt eller buktiga ytor. Vill vi approximera buktiga ytor låter vi varje hörnpunkts normal vara medelvärde av de trianglar som delar på punkten. Men detta är ju direkt olämpligt för objekt som avses vara flata, t ex en kub, eftersom det genast blir konflikt om två eller flera trianglar delar på en hörnpunkt. För flata objekt vore det alltså lämpligare att associera normalen med triangeln, inte med dess hörnpunkter.

En acceptabel kompromiss är att låta varje hörnpunkt ha en normal och istället kräva att PLG-filerna för *flata objekt* är definierade så att hörnpunkterna inte delas (se fig. 7).



Figur 7: Transformation för flata objekt.

Detta gör det möjligt för oss att approximera buktiga ytor och kommer dessutom att underlätta tilldelningen av texturkoordinater i nästa laboration. Kuben till höger i fig. 7 finns tillgänglig på kursens hemsida som `sbox.plg`.

Lägg därför till följande fält i `Vertex`, i filen `object.h`:

```
typedef struct vertex {  
    :  
    Vector normal;  
    double diff, spec;  
} Vertex;
```

Dessa fält kommer att innehålla nodens normal samt diffusa och speglande komponent, respektive.



Figur 8: Exempel på en buktig yta: `pot.plg`. Observera att man ser rakt igenom kannan där baksidor har eliminerats, t ex längst ut på pipen.

Grundläggande funktioner. För att erhålla full poäng på laborationen måste ni implementera de grundläggande funktionerna som beskrivs nedan (50 poäng). Om ni vill kan ni dessutom genomföra några av de valfria alternativen, för extra poäng.

- **Normaler** (10 p).

Skriv en rutin som interpolerar normalerna så att en nods normal representerar medelvärde för de trianglar som delar på noden. Anropa denna rutin när ett objekt laddats in.

För att förenkla belysningsberäkningarna kan det vara en god idé att låta normalerna vara enhetsvektorer.

- **Phong-lighting** (40 p).

Skriv proceduren `shade(Vertex *p)` som "tänder" en nod genom att räkna ut fälten `p->diff` och `p->spec`.

Använd Phong's belysningsmodell med en ensam punktformad ljuskälla. Ljuskällan kan antingen

placeras på en fix position eller följa med observatören.

Ni kan låta reflektionskoefficienterna k_d och k_s , samt dämpningsfaktorn $f(d)$, vara 1. Detta gör att de olika komponenterna i belyningsmodellen blir

- $I_{diff} = I_p \max(0, L \cdot N)$.
- $I_{spec} = I_p \max(0, R \cdot V)^n$, om² $L \cdot N > 0$.

Anropa `shade` från er pipeline för att tända hörnpunkterna i en triangel innan den svepkonverteras. Interpolera sedan I_{diff} och I_{spec} över triangeln som ni nu gör med z -värdet. Den totala intensiteten i varje pixel innehåller även bakgrundsbelysning, dvs

$$I = I_a + I_{diff} + I_{spec}.$$

(Anledningen till att vi inte räknar ut I för varje hörnpunkt och sedan interpolerar endast detta värde beror på att vi i nästa laboration kommer att modulera den diffusa reflektionen med en textur.)

Extra funktioner. Flera bonuspoäng kan införskaffas genom att lösa några av följande uppgifter.

- *Phong-shading* (20 p).

För stora trianglar kommer Gouraud-shading att missa ”blänket”, såvida det inte hamnade på en hörnpunkt. Lösningen på detta problem är att interpolera normalen över triangeln och beräkna intensiteten i varje pixel.

Implementera pseudokoden för Phong-shading från föreläsningen. Som ni kommer att märka är denna metod ganska tidskrävande och det kan därför vara lämpligt att visa objekten som trådmodeller när man förflyttar sig i världen och låta användaren trycka på en knapp när belysning önskas.

- *Flera ljuskällor* (10 p).

En naturlig utvidgning är att tillåta flera ljuskällor. Dock ger detta problem när den beräknade intensiteten överskrider maxvärdet 1. För att få full poäng på denna uppgift vill vi därför att ni *inte* endast kapar värden > 1 , utan skalar om intensiteterna så som diskuterats på föreläsningen.

²Glöm inte `#include <math.h>` om ni använder `pow` i beräkningen av speglade reflektion!

- *Färg* (10 p)

En naturligt utvidgning är att låta ljuskällan ha olika grader av röd, grön och blå färg. Låt sedan objekten ha olika reflektionskoefficienter för dessa färger. Koefficienterna kan vara attribut i `Object`. Beräkna sedan varje färg för sig när hörnpunkterna tänds.

- *Spotlight* (10 p)

Läs om hur spotlights fungerar i OpenGL och implementera sedan denna funktionalitet i ert program.

- *Skuggor* (40 p).

Läs om och implementera någon metod för skuggkastning ur boken.

- *Mer bonusuppgifter?*

Kom ihåg att ni kan hitta på egna utökningar, även för OpenGL-versionen.

5. Texturmappning

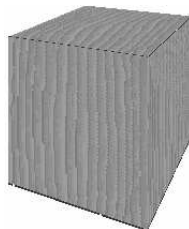
Förberedelser. Repetera föreläsningssanteckningarna om texturmappning, samt bokens avsnitt 14.3.2.

Uppgift. Som en sista “feature” i vår pipeline kommer vi att implementera 2D texturmappning, dvs möjligheten att klistra på en godtycklig textur på ett objekts ytor.

Återigen finns en nedbantad OpenGL-version av laborationen att ladda ner från hemsidan. Den visar upp en kub med MdH:s logotyp som texturer på sidorna. Beskrivningen av kubens sidor, normaler och texturkoordinater är hårdkodade i GL-programmet. Vi kommer dock som vanligt att läsa in geometrin från en PLG-fil. Normalerna interpolerar ni fram precis som förut. Texturmappning specificeras i PLG-filen för objektet (se fig. 9) genom att:

1. Ange filnamnet på den textur som ska användas. Denna fil är en skrå PPM-fil och kan skapas från andra format med bl a Linux-verktygen `xv` eller `gimp`. På kursens hemsidan finns ett antal PPM-filer att hämta. För att läsa in PPM-filer finns filerna `ppm.c` och `ppm.h` också att hämta från hemsidan, tillsammans med en ny `Makefile` som ser till att PPM-läsaren kompileras ihop med resten av programmet.
2. Specificera texturkoordinater för varje nod. Dessa värden ligger normalt i intervallet $0 \dots 1$, där $(0,0)$ och $(1,1)$ motsvarar nedre vänstra hörnet respektive övre högra hörnet i texturen.

```
TBOX 24 6 wood.ppm
100 100 -100 1.0 0.0
-100 100 -100 0.0 0.0
-100 100 100 0.0 1.0
100 100 100 1.0 1.0
100 100 -100 1.0 1.0
100 -100 -100 1.0 0.0
-100 -100 -100 0.0 0.0
-100 100 -100 0.0 1.0
```



Figur 9: Exempel på texturmappning: `tbox.plg`.

Grundläggande funktioner. Denna laboration har endast en grundläggande funktion:

- *Texturmappning* (50 p).

Typdefinitionen för `Texture` i `defs.h` specificerar en textur så att `image[y * width + x]`,

$0 \leq x < \text{width}, 0 \leq y < \text{height}$, ger ett intensitetsvärde $0 \dots 255$.

```
typedef struct texture {
    int height, width;
    Byte *image;
} Texture;
```

Börja med att lägga till ett `Texture`-fält i definitionen av `Obj` i `object.h`:

```
typedef struct obj {
    Texture *texture;
    :
} Obj;
```

Inläsning av PPM-filen för ett objekt görs automatiskt av `load` i `object.c` som nu har ändrat beteende pga definitionen av symbolen `TEXTURE` i `Makefile`.

Ni behöver också deklarerat texturkoordinaterna för noder. (Även dessa läses in automatiskt av `load`.)

```
typedef struct vertex {
    :
    double s,t;
} Vertex;
```

Det som återstår att göra nu är att implementera Heckbert och Moretons algoritm, förslagsvis så som den presenterats på föreläsningen. Observera att man kan använda en `Vector` för att lagra en homogen 2D-vektor (s, t, q) .

Ni får själva välja om ni vill implementera repetition eller “clamping” om texturkoordinaterna ligger utanför intervallet $0 \dots 1$.

Ni kan också välja mellan dekal-mode, dvs

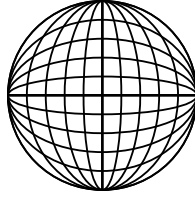
$$I = \text{texel}(s, t),$$

vilket ignorerar intensiteten från föregående laboration, eller modulering av texturinformatjonen med intensiteten, t ex

$$I = (I_a + I_{diff}) \cdot \text{texel}(s, t) / 256.0 + I_{spec}.$$

Extra funktioner. Flera bonuspoäng kan införskaffas genom att lösa några av följande uppgifter.

$$\begin{aligned}x(s, t) &= r \sin(\pi s) \cos(2\pi t) \\y(s, t) &= r \sin(\pi s) \sin(2\pi t) \\z(s, t) &= r \cos(\pi s)\end{aligned}$$



Figur 10: Sfärisk yta parametriserad map $s, t \in [0, 1]$.

- *Texturmappad sfär* (10 p).

Skriv ett program som genererar (tesselerar) en sfär och tillhörande texturkoordinater för dess noder. Använd den parametriserade formen i fig. 10 så att s och t representerar latituder respektive longituder.

Ni behöver inte ta hänsyn till de diskontinuiteter som uppstår vid polerna eller "sömmen" då t ex s gör ett språng från 1 tillbaks till 0.

- *3D texturmappning* (20 p).

Implementera 3D texturmappning så som den beskrivits på föreläsningen. Om ni interpolerar världskoordinaterna linjärt istället för i perspektiv uppstår perspektivfel vilket endast ger 10 poäng.

- *Material* (10 p).

Olika material, t ex marmor, kan simuleras med olika noise-funktioner. Material för denna extrauppgift finns att hämta på kurssidan.

- *Bump-mapping* (40 p).

Denna uppgift är lämplig om ni gjort extrauppgiften för Phong-shading från förra laborationen.

Bump-mapping innebär att texturen ger upphov till små förändringar av ytnormalen, vilket leder till ett intryck av gropighet. Lämplig litteratur för denna extrauppgift finns på kurssidan.

- *Mer bonusuppgifter?*

Kom ihåg att ni kan hitta på egna utökningar, även för OpenGL-versionen.